

Polynomial-time algorithms for special cases of the maximum confluent flow problem[☆]

Daniel Dressler^a, Martin Strehler^{b,*}

^a TU Berlin, Inst. f. Mathematik, MA 5-2, Str. des 17. Juni 136, 10623 Berlin, Germany

^b BTU Cottbus, Mathematisches Institut, Postfach 10 13 44, 03013 Cottbus, Germany

ARTICLE INFO

Article history:

Received 26 October 2010

Received in revised form 5 April 2013

Accepted 5 July 2013

Available online 29 July 2013

Keywords:

Confluent flow

Outerplanar graph

Polynomial-time algorithm

Treewidth

FPTAS

ABSTRACT

A flow on a directed network is said to be confluent if the flow uses at most one outgoing arc at each node. Confluent flows arise naturally in destination-based routing. We study the maximum confluent flow problem (MAXCONF) with a single commodity but multiple sources and sinks and heterogeneous arc capacities. It was recently shown that MAXCONF is $\mathcal{NP}$ -hard even on trees. We improve the classification of easy and hard confluent flow problems by providing polynomial-time algorithms for outerplanar graphs with a single sink, as well as trees with a constant number of either sources or sinks. Furthermore, we present an FPTAS for graphs with bounded treewidth.

© 2013 Elsevier B.V. All rights reserved.

1. Introduction

We see them every day, these green pictograms with the little stick figure fleeing from danger: emergency exit signs. Creating evacuation plans means, besides lots of other tasks, choosing the escape routes. And here arises a crucial point. Two persons arriving at the same evacuation sign will follow the same route. A group of people fleeing together will usually not split. On the other hand, people should be partitioned evenly among the different emergency exits to avoid congestion.

Network flows have been a major tool for problems like these ever since Ford and Fulkerson [9] studied them. But a good evacuation plan cannot be created by simply solving a maximum flow problem. Instead, we have to consider a so-called confluent flow. A flow on a directed network is said to be confluent if the flow uses at most one outgoing arc at each node. This has far-reaching consequences. Flow-carrying paths may not cross or split, and every source is assigned to exactly one sink. The total view shows the flow-carrying arcs forming an oriented subforest of the graph, pointing towards the sinks. Not surprisingly, finding an optimal confluent flow is a hard problem.

Confluent flows arise in a number of scenarios besides evacuations. Destination-based routing on the Internet is perhaps the most common application of a multi-commodity confluent flow [2]. Railway logistics may contain a confluent flow subproblem as well [10]. Another scenario we are interested in is inner-city traffic coordination. Non-local road users will follow the destination signs and users of satellite navigation systems will be almost always guided on the same route when they have the same destination. Therefore, when modeling such traffic, one has to consider a multi-commodity flow problem with some confluent commodities.

In this article, we consider two types of confluent flow problem. In the maximum confluent flow problem (MAXCONF), the goal is to send the maximum amount of flow from the sources to the sinks in a confluent way. The confluent transshipment problem (CONFTRANS) is a decision problem about whether the entire given amounts of flow units can be routed confluent

[☆] Supported by the Federal Ministry for Education and Research (BMBF) under grant 03KOPAI3 and 03SKPAI6 within the Advest project.

* Corresponding author. Tel.: +49 355692981; fax: +49 355693164.

E-mail address: strehler@math.tu-cottbus.de (M. Strehler).

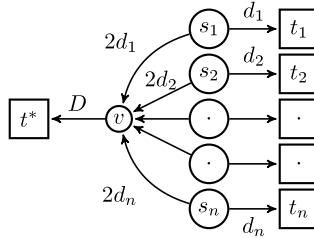


Fig. 1. This tree models a PARTITION instance using n sources $\{s_i\}$, and $D = \sum_{i=1}^n d_i$. Labels at arcs refer to capacities. The optimum confluent flow value is $\frac{3}{2} \sum_{i=1}^n d_i$ if and only partitioning $\{2d_i : 1 \leq i \leq n\}$ into two sets of equal sum is possible.

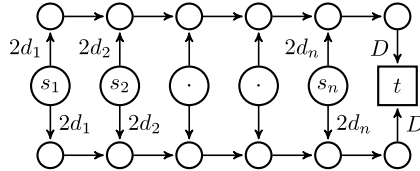


Fig. 2. The modified PARTITION instance of Fig. 2 with a single sink. This instance implies that MAXCONF with one sink is at least weakly $\mathcal{NP}$ -hard on $3 \times n$ grids, series-parallel graphs (or graphs with treewidth 2), and 2-outerplanar graphs, all of which are in particular planar.

to the sinks. The notable difference to previous works is that the flow has to obey heterogeneous arc capacities. Consequently, not all supplies may be routable. Furthermore, the sources and sinks may have upper bounds on their supplies or demands.

Literature overview

Despite various applications, pure confluent flow problems are rarely studied in literature. Chen et al. [5,6] introduce confluent flows and study the objective of minimizing the maximum node congestion (i.e., the maximum flow into a node, which always occurs at a sink). They also derive an inapproximability result in the case of uniform capacities. For heterogeneous arc capacities, Dressler and Strehler [8] show that it is $\mathcal{NP}$ -complete to approximate MAXCONF with performance ratio better than $3/2$ on general graphs. They also provide a fully polynomial-time approximation scheme for graphs with treewidth bounded by a constant. Furthermore, they prove that MAXCONF is (weakly) $\mathcal{NP}$ -hard even on trees if multiple sinks are allowed and on graphs with treewidth 2 and a single sink (see Figs. 1 and 2 for an idea of the reduction).

In a series of papers, Fujishige et al. [11] consider confluent flows over time with multiple sinks, but restricted to trees. Interestingly, this problem can be solved in polynomial time even though the related static MAXCONF problem cannot. One key difference is that arc capacities in flows over time limit the flow rate and not the total flow on an arc, as in the static version. Thus, more flow can always be achieved at the expense of more time, whereas a static flow problem simply becomes infeasible if the given capacities are insufficient.

Our results

In this paper, we present new results on graph classes for which confluent flow problems are efficiently solvable. For the single-sink case, we reduce the gap between the trivial case of a tree and the $\mathcal{NP}$ -hard case of treewidth 2 by providing a polynomial-time algorithm for outerplanar graphs. We also refine the complexity analysis for trees, showing that even with multiple sources and sinks, CONFTRANS is solvable in polynomial time. In contrast, the closely related optimization problem MAXCONF is $\mathcal{NP}$ -hard, but we show that it is polynomial-time solvable if the number of either sources or sinks is constant. We complete the analysis by providing an improved version of our FPTAS for MAXCONF on graphs with bounded treewidth presented in [8].

Flow problems are rarely studied on graphs with bounded treewidth or outerplanar graphs. Unlike other combinatorial problems, flows involve more calculations, e.g. for flow conservation. When one may consider only small parts of a network, one has to deal with an unknown amount of incoming and outgoing flow. This significantly complicates dynamic programming. Consequently, most ideas in this paper focus on compressing or reducing data such that confluent flows can be computed efficiently.

Overview of the paper

This paper is organized as follows. In Section 2, we provide basic definitions. Confluent flows on trees and graphs with bounded treewidth are discussed in Section 3. Afterwards, we show in Section 4 that the problem of determining a maximum confluent flow on outerplanar graphs can be solved in polynomial time. Finally, in Section 5, we discuss the remaining gap between easy and hard problems for confluent flows.

2. Preliminaries

Let $G = (V, A)$ denote a directed graph without multiple arcs or loops. We use $A(G)$ to denote the arc set of a graph. For $V' \subseteq V$, let $G[V']$ denote the subgraph of G induced by V' , and let $A[V'] := A(G[V'])$. For $v \in V$, let $\delta^+(v) := \{(w', w'') \in$

$A : w' = v\}$ and $\delta^-(v) := \{(w', w'') \in A : w'' = v\}$ be the outgoing and incoming arcs of v , respectively. The underlying undirected graph $\bar{G}$ of a digraph $G = (V, A)$ is $\bar{G} = (V, E)$ with $E := \{(v, w) : (v, w) \in A\}$.

Definition 1 (Instance). An instance of a flow problem consists of a directed graph $G = (V, A)$, a capacity function $u : A \rightarrow \mathbb{R}_{\geq 0}$, and a supply–demand function $d : V \rightarrow \mathbb{R}$. The vertices $S := \{v \in V : d(v) > 0\}$ are called *sources*, while $T := \{v \in V : d(v) < 0\}$ are the *sinks*.

Definition 2 (Flow). Given an instance, a *flow function* is a function $x : A \rightarrow \mathbb{R}$ with $0 \leq x(a) \leq u(a)$ for all $a \in A$. A flow function is *integral* if it has only integral values. The *inflow* and *outflow* of a flow function x at a given node v are $\text{inflow}_x(v) := \sum_{a \in \delta^-(v)} x(a)$ and $\text{outflow}_x(v) := \sum_{a \in \delta^+(v)} x(a)$. Let $\text{bal}_x(v) := \text{outflow}_x(v) - \text{inflow}_x(v)$ be the *balance* of a node. A flow function x is a *flow* if it satisfies the *flow conservation constraints* $0 \leq \text{bal}_x(v) \leq d(v)$ for all $v \in V \setminus T$ and $0 \geq \text{bal}_x(t) \geq d(t)$ for all $t \in T$. The *value* of a flow is $\text{val}(x) := -\sum_{t \in T} \text{bal}_x(t)$.

Definition 3 (In-Forest). A graph $G = (V, A)$ is an *in-forest* if each vertex has at most one outgoing arc and G contains no directed cycle.

It is easy to see that an in-forest never contains an undirected cycle either. A *confluent flow* is a flow such that its flow-carrying arcs form an in-forest and no flow leaves any sink.

Definition 4 (Nearly Confluent Flow, Confluent Flow). For an instance and a flow x , let $F_x := \{(v, w) \in A : x(v, w) > 0\}$ be the set of flow-carrying arcs. A flow x is called *confluent* if F_x is an in-forest and $F_x \cap \delta^+(t) = \emptyset$ for all $t \in T$. A flow x is called *nearly confluent* if $|F_x \cap \delta^+(v)| \leq 1$ for all $v \in V$ and $F_x \cap \delta^+(t) = \emptyset$ for all $t \in T$.

Note that one could also broaden this definition to allow flow to leave sinks again. This would further complicate some results, because then one cannot readily make use of a supersink, as is often done for flows. In a way, such sinks are also not “confluent” because flow units previously traveling together are separated at such sinks.

We can now formally define MAXCONF and CONFTRANS.

Problem 1 (CONFTRANS). INPUT: An instance G, u , and d .

OUTPUT: YES if and only if there is a confluent flow of value $\sum_{s \in S} d(s)$.

Problem 2 (MAXCONF). INPUT: An instance G, u , and d .

OUTPUT: The maximum flow value over all confluent flows.

Clearly, an algorithm for solving MAXCONF can solve CONFTRANS as well by comparing the flow value to $\sum_{s \in S} d(s)$.

One helpful way to view confluent flows and the associated MAXCONF and CONFTRANS is to break the flow computation into two parts. The first step is to determine the set of arcs that actually carry flow, precisely the in-forest F_x . Once F_x is fixed, one can use any maximum (standard) flow algorithm to compute an optimal confluent flow on F_x in a second step. (Since F_x is a forest, this subproblem is even easier than a standard flow computation.) Thus, solving any confluent flow problem essentially reduces to picking the optimum in-forest. One important consequence is that maximum confluent flows can always be chosen integral if the input is integral, because they can be computed as maximum flows, which are well known to have integral solutions under this assumption. In the remainder of this contribution, we will only consider integral input, which also accounts for rational input after scaling.

3. Trees and graphs with treewidth bounded by a constant

3.1. Confluent transshipments on trees

MAXCONF on a tree with a single sink is an easy problem, because the optimal in-forest is obvious. But, as shown in [8], MAXCONF with multiple sinks is weakly $\mathcal{NP}$ -hard even on trees. Seemingly in contrast to this, CONFTRANS on trees can be solved in polynomial time, and we will use a finding of Chen et al. [6] to prove this in Theorem 1. It is important to note that one cannot use an algorithm for CONFTRANS together with binary search on the flow value to solve MAXCONF because there are multiple sources and the supply of each source would have to be specified exactly. Aside from this, we will show in this section that there exist efficient algorithms for MAXCONF when either the number of sinks or the number of sources is bounded by a constant. Additionally, instances where the underlying undirected graph only differs from a tree by a constant number of edges can be reduced to a tree by “guessing” a suitable tree, i.e., with brute force. Finally, we present an improved fully polynomial-time approximation scheme for MAXCONF on graphs with treewidth at most k .

We first consider CONFTRANS for an instance with multiple sinks, where $G = (V, A)$ is a bidirected tree. That is, the underlying undirected graph is a tree, and anti-parallel arcs may exist. Chen et al. [6] studied confluent flows on trees with congestion and uniform capacities. They present a polynomial-time algorithm which can be extended to an algorithm for CONFTRANS with arbitrary capacities by slight modifications. The main idea is to resolve the confluent flow constraints at the leaves of the tree. For example, if there is a leaf that is a source node, its supply must leave along the only arc to the parent vertex.

Theorem 1. *There exists a polynomial-time algorithm for CONFTRANS on bidirected trees (with arbitrarily many sources and sinks).*

Proof. We assume that the tree is rooted at an arbitrary vertex. The algorithm proceeds to delete leaf after leaf, each time yielding a smaller equivalent instance of CONFTRANS, until only a single vertex remains. The instance is a Yes-instance if and only if this vertex has non-positive supply.

Suppose that there is a leaf v with $d(v) \geq 0$. Let w denote the parent of v . All flow of v must be routed to w . If $u(w, w) < d(v)$, the flow cannot be routed, and the instance is a No-instance. Otherwise, we can increase $d(w)$ by $d(v)$ and delete v .

In the remaining case, all leaves have negative supply. We want to consider a vertex w in the second-lowest layer of the tree. All its children are leaves and therefore sinks. Let v^* be the leaf below w that maximizes $\min\{u(w, v), -d(v)\}$, i.e., v^* is the best sink among the children of w . Its effective capacity is $c^* := \min\{u(w, v), -d(v)\} \geq 0$.

If $d(w) \leq c^*$ (in particular, if w is not a source), there is no need to route flow from w out of its subtree, so we can contract the arc (w, v) . In terms of the algorithm, we set $d(w) := d(w) - c^* \leq 0$ and delete all the children of w . On the other hand, if $d(w) > c^*$, there is no possibility to route all the flow from w except upwards, so we leave $d(w)$ unchanged but again delete all children of w . $\square$

Next, we show the existence of polynomial-time algorithms for MAXCONF if either the number of sources or the number of sinks is bounded. Recall that MAXCONF on trees with an unlimited number of sources and sinks is $\mathcal{NP}$ -hard in general. Let the number of sources be bounded by a constant k . In a confluent flow, the flow from each source travels along a path that eventually ends at one sink. As there are at most $|V|^k$ assignments of the sources to the sinks, and the paths taken by the flow units are uniquely determined, all possibilities can simply be enumerated and checked in polynomial time. Similarly, if the number of sinks is bounded by a constant k , then the solution can be characterized by a cut consisting of at most $k-1$ arcs that leaves only one sink in each resulting component. These cuts can be enumerated in $\mathcal{O}(|V|^k)$. This yields the next theorem.

Theorem 2. *MAXCONF on trees can be solved in polynomial time if either the number of sinks or the number of sources is bounded by a constant.*

3.2. MAXCONF on graphs with bounded treewidth

We now want to present an approximation scheme for MAXCONF that works for graphs with bounded treewidth. As the hardness result in [8] shows, we cannot approximate MAXCONF with arbitrary precision on general graphs (unless $\mathcal{P} = \mathcal{NP}$), so such a restriction to certain graph classes is necessary. On the positive side, the treewidth hierarchy eventually covers all graphs, so the algorithm at least works on all graphs, but the running time grows exponentially with the treewidth.

Here, we present an improved and simplified version of the approximation scheme in [8]. However, we omit the very technical proofs of most of the lemmata. The full proofs can be found in [7]. The first step towards the approximation scheme is a dynamic program as usual for graphs with bounded treewidth. For certain subgraphs, we want to determine all feasible combinations of incoming and outgoing flow values. However, we cannot efficiently encode these tables when we consider graphs with treewidth 2. This follows from the PARTITION instances in Fig. 2. There are pseudo-polynomially many possible combinations of flow values on the two arcs leading towards the sink. A dynamic program must account for all of these if the capacity of the following arcs is not yet known. Therefore, we round the dynamic program to polynomially many entries in a second step to achieve an approximation scheme.

Note that all combinatorial problems that can be described by extended monadic second-order logic (EMS) can be solved in polynomial time on graphs with bounded treewidth. See Arnborg et al. [1] for these results and sample lists of combinatorial problems which this covers. Since flow conservation needs some counting, this does not immediately solve our problem (which remains $\mathcal{NP}$ -hard on graphs with treewidth 2), but a reduction to an instance of *pseudo-polynomial* size could be possible. This instance could then be solved in effectively pseudo-polynomial time. This would remove the theoretical need for the first step above, that is, developing a pseudo-polynomial dynamic program. But using the results from EMS typically yields weak bounds on the running times and reveals little about the structure needed to actually implement the dynamic program efficiently. In particular, we think understanding the structure of the dynamic program is important for those wishing to prove Conjecture 1.

For defining treewidth, we follow Bodlaender [3], and we consider a version that only depends on the underlying undirected graph. To deal with tree decompositions algorithmically, it is helpful to strengthen the definition to allow only few specialized “nice” cases. The idea is that the bags should change by at most a single vertex while traversing the superimposed tree towards the designated root, which will be the single sink of the confluent flow problem. Such an approach is commonly used, but the exact definitions differ slightly from author to author.

Definition 5 ((Nice) Tree Decomposition). Let $G = (V, E)$ be an undirected graph. A *tree decomposition* of G is a tree $(I, \mathcal{T})$ on some new node set I with edges $\mathcal{T}$, and a family of sets $B_i \subseteq V$ for $i \in I$, such that the following three properties hold.

1. The B_i cover V ; that is, $\bigcup_{i \in I} B_i = V$.
2. For every edge $\{u, v\} \in E$, there is an $i \in I$ with $\{u, v\} \subseteq B_i$.
3. For every $v \in V$, the set of all $i \in I$ such that $v \in B_i$ forms a connected subgraph of $(I, \mathcal{T})$.

Given a root $r \in I$ of the tree and a vertex $t \in V$, a *nice tree decomposition* additionally must satisfy the following.

4. If a node $i \in I$ has two children $j, k \in I$, then $B_i = B_j = B_k$. In this case, i is called a **MERGE** node.
5. If a node $i \in I$ has one child j , then one of the following must hold.
 - (a) $|B_i| = |B_j| - 1$ and $B_i \subseteq B_j$. Then i is called a **FORGET** node.
 - (b) $|B_i| = |B_j| + 1$ and $B_j \subseteq B_i$. Then i is called an **INTRODUCE** node.
6. $|B_i| = 1$ holds for all leaves i of $(I, \mathcal{T})$. Such an i is called a **LEAF** node.
7. $B_r = \{t\}$ holds.

Each B_i is called a *bag*. The *width* of a tree decomposition is $\max_{i \in I} |B_i| - 1$. The *treewidth* of G is the minimum k such that G has a tree decomposition of width k .

In general, it is $\mathcal{NP}$ -hard to compute the treewidth of a graph. However, if the treewidth is known to be bounded by a constant k , many related problems can be solved in polynomial or even linear time. In particular, one can check in linear time whether a graph has treewidth at most k . One can also construct a tree decomposition with width k in linear time, if it exists. But the hidden factors are large and grow quickly with k . An overview of achievable running times and algorithms is given in [3]. Given a tree decomposition of $G = (V, E)$, it is simple to construct a nice tree decomposition by adding intermediate bags. We omit the technical details here. They can be found in [7].

Lemma 1. *Let $G = (V, E)$ be a graph of treewidth k , and let $t \in V$ be an arbitrary vertex. Then G has a nice tree decomposition $(I, \mathcal{T})$ with bags $(B_i)_{i \in I}$ with width k and with root r such that $B_r = \{t\}$. Furthermore, the number of nodes in I is polynomially bounded in $|V|$, and the decomposition can be constructed in linear time, assuming that k is constant.*

This yields the following properties, which we will exploit in the dynamic program.

Lemma 2. *Let $(I, \mathcal{T})$ with bags $(B_i)_{i \in I}$ be a nice tree decomposition of $G = (V, E)$ with root r .*

1. *For every vertex $v \in V \setminus B_r$, there is a unique FORGET node $i \in I$ with child $j \in I$ such that $B_j = B_i \cup \{v\}$. There is no such FORGET node for $v \in B_r$.*
2. *For every MERGE node i with children j and k , it holds that $B_i = V(G_j) \cap V(G_k)$.*
3. *For every INTRODUCE node i with child j , let v be the unique vertex in $B_i \setminus B_j$. Then there are no edges between v and $V(G_j) \setminus B_j$.*

Because a nice tree decomposition designates a root, we can consider the bags of the subtree rooted at some $i \in I$ (which also includes i itself). The vertices contained in these bags induce a subgraph G_i of G , which we call the *bagend*. Algorithmically, the bagends are the subgraphs on which we want to compute intermediate solutions, with the bagend of the root being the entire graph. The bag B_i is called the *interface* between G_i and G , because it can be seen that any path from $V(G_i)$ to $V(G) \setminus V(G_i)$ must pass through B_i .

Definition 6 (*Bagend*). Consider a nice tree decomposition $(I, \mathcal{T})$ with bags $(B_i)_{i \in I}$. For $i \in I$, the *bagend* G_i is the subgraph of G induced by the union of all B_j for j in the subtree rooted at i .

We first restrict ourselves to the single-sink case of MAXCONF. We will generalize this when we describe the approximation algorithm. We are given a graph $G = (V, A)$ with the sink t , and can assume that we also have a nice tree decomposition with a bag only containing t as root, as shown in Lemma 1.

We work bottom-up from the LEAFs of the tree decomposition to the root containing t . The bags and bagends of a nice tree decomposition change towards the sink in an orderly fashion, introducing and forgetting one vertex at a time. So this approach is fairly straightforward if we have suitable representations of confluent flows on a bagend G_i that we can express just by information on the interface $G[B_i]$. When we have done this for all bags, it just remains to read off the maximum flow value at the root.

The arcs chosen in B_i and the flow sent on them seem like natural candidates for the dynamic program, but this is not the choice we will make, because finding a tree-like flow on the tree-like structure has not as many synergies as one might think. Sources may send flow from near the sink t through the interface B_i to G_i , then out of B_i and to the sink near where the flow started; see Fig. 3. So when we consider the restriction of a confluent flow to G_i , we need to account for sources we have not yet encountered. We do this by ignoring the balance condition of the vertices in B_i and tracking the outgoing and incoming flow at each vertex separately.

We try to make the technicalities a bit simpler by fixing the outflow of a vertex once and for all when an outgoing arc is chosen for it. We will denote this *target outflow value* by $o(v)$. Any imbalance must be made up for by the supply of the vertex and the incoming flow. As we collect incoming flow during the run of the algorithm, the sources of this flow might no longer be visible, and therefore we think of this quantity as the *additional supply* $d^+(v)$ of v . The main advantage of fixing the target outflow is that we can locally manipulate the set of flow-carrying arcs. For example, if a vertex v uses no outgoing arc so far, and we want to send flow along an arc (v, w) , we can simply do so without triggering a chain reaction of increasing flow along the path that these flow units take. Indeed, such a path could lead into already processed parts of the bagend that are no longer in the interface, which would make tracking the changes even harder. Instead, we simply increase the additional supply $d^+(w)$ of the receiving vertex. Its target outflow $o(w)$ remains unchanged, and therefore no further steps need to be taken.

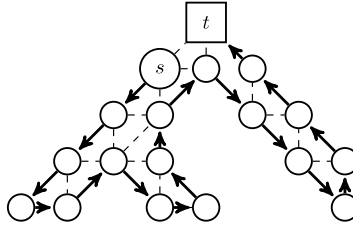


Fig. 3. On a tree, confluent flow always moves towards the sink. On a graph with bounded treewidth (in particular, the displayed graph has treewidth 2), flow may also move “away” from the sink taking a detour through the branches.

Regarding arcs, note that we do not need to consider the chosen arcs of $A[B_i]$ in the dynamic program. Instead, we assume that no decision has been made about the arcs in the current bag. We only consider whether a vertex has no outgoing arc yet, in which case its target outflow value is 0; or it already has an outgoing arc, in which case $o(v)$ must be positive. When a vertex leaves a bag due to a FORGET node in the tree decomposition, all vertices in the bag with no outgoing flow have the opportunity to choose an arc towards the leaving vertex. If the leaving vertex had no outgoing flow itself, it may also choose one arc pointing to a vertex in the bag. So only then are the arcs fixed, but they are no longer visible in the arc set of the interface. Eventually, the bags considered converge to the root bag, which only contains the sink t and no arcs, so all vertices can choose their outgoing arc at some point.

One final note is that we cannot avoid directed cycles in such a localized approach, so, instead of considering confluent flows, we consider nearly confluent flows. Recall from Definition 5 that such a flow is simply a flow in which each vertex has at most one outgoing arc and sinks have none, and that considering nearly confluent flows will give us the same maximum flow value as MAXCONF.

These considerations motivate the following table for each node of the tree decomposition.

Definition 7 (Table). Let $G = (V, A)$ be a graph with arc capacities $u : A \rightarrow \mathbb{Z}_{\geq 0}$ and supplies $d : V \rightarrow \mathbb{Z}$, but only with a single unlimited sink t , which in addition must satisfy $\delta^{\text{out}}(t) = \emptyset$. Let U be an upper bound on MAXCONF for this instance, and assume that $u(a) \leq U$ for all arcs $a \in A$.

Let $(I, \mathcal{T})$ and $(B_i)_{i \in I}$ be a nice tree decomposition of G . For $i \in I$, let $\text{TABLE}(i)$ be defined as the set containing exactly the elements (o, d^+) with $o \in \{0, 1, \dots, U\}^{B_i}$, $d^+ \in \{0, 1, \dots, U\}^{B_i}$ for which there is an integral feasible nearly confluent flow function x in G_i with the following properties.

1. $x(a) = 0$ for all $a \in A[B_i]$.
2. $o(v) = \text{outflow}(v)$ for all $v \in B_i$.
3. $d^+(v) = \text{inflow}(v)$ for all $v \in B_i$.
4. $0 \leq \text{bal}(v) \leq d(v)$ for all $v \in V(G_i) \setminus B_i$.

Please note that, given a flow function x on the bagend G_i , it is easy to check whether x satisfies all conditions. Also, x determines a unique entry $(o, d^+) \in \text{TABLE}(i)$.

We finish the description of the dynamic programming table with a lemma showing that this table contains the solution to MAXCONF at the root node of the tree decomposition.

Lemma 3. Under the assumptions of Definition 7, the optimum value of MAXCONF is

$$\max_{(o, d^+) \in \text{TABLE}(r)} d^+(t),$$

where $r \in I$ with $B_r = \{t\}$ is the root node of the nice tree decomposition.

Proof. Let f^* be the optimum value of MAXCONF. An optimum confluent flow for this instance can be chosen integral and implies an entry $((0), (f^*))$ in $\text{TABLE}(r)$. Condition 1 on the arcs is trivially satisfied, Conditions 2 and 3 on the only vertex t in B_r just correspond to the entry, and a confluent flow satisfies the balance conditions at all vertices v in $G_r \setminus B_r = V \setminus \{t\}$ as well, which satisfies Condition 4.

On the other hand, any entry $(o, d^+) \in \text{TABLE}(r)$ implies some integral feasible nearly confluent flow function x on $G_i = G$. Trivially, $o(t) = 0$ must hold, because $\delta^{\text{out}}(t) = \emptyset$. Then x is a flow, because the balance conditions are satisfied at all non-sink vertices by Condition 4, and the sink has infinite demand and no outgoing flow. Then $d^+ = (\text{inflow}(t)) = (\text{val}(x))$ is indeed the flow value of a nearly confluent flow, which attains the same maximum as a confluent flow. $\square$

3.3. Computing the tables

For computing the entries of $\text{TABLE}(i)$ from the tables of the children of i , subroutines are needed that address the four classes of nodes (LEAF, INTRODUCE, MERGE, and FORGET). One has to ensure that the flow values are calculated correctly, and to prove that all entries can be derived in this way. These steps are mostly technical, and the main work is done in the FORGET nodes, the only place where the algorithm has to decide which arcs should carry flow. All of the lemmata detailing these constructions operate under the assumptions and notation of Definition 7 without explicitly saying so every time.

We begin with the trivial case of a LEAF node. The bagend consists of just a single vertex and has no arc, so all flow functions are trivial, and the only possible entry corresponds to no incoming or outgoing flow.

Lemma 4. For a LEAF node i , $\text{TABLE}(i) = \{((0), (0))\}$.

Proof. The bagend G_i consists of a single vertex only, and in particular contains no arcs. So any x has to be an empty function, and Conditions 1 and 4 do not apply. The other conditions simply force an entry to be $((0), (0))$. Since these are all conditions, this is a valid entry. $\square$

In an INTRODUCE node, the new vertex is simply added with target outflow 0 and additional supply 0 while leaving all other entries unchanged. (As mentioned after the definition of a nice tree decomposition, the LEAF nodes could also be considered as INTRODUCE nodes that add a vertex to an empty bag, but this would be an atypical setup.)

Lemma 5. Let i be an INTRODUCE node, and let j be the unique child of i . Let v be the new vertex, such that $B_i = B_j \cup \{v\}$. Let $o, d^+ \in \{0, \dots, U\}^{B_i}$.

Then $(o, d^+) \in \text{TABLE}(i)$ if and only if $o(v) = d^+(v) = 0$ and $(o|_{B_j}, d^+|_{B_j}) \in \text{TABLE}(j)$.

As said above, the proof for this and the following lemmata can be found in [7]. When dealing with MERGE nodes, all bags contain the same vertices but the bagends differ: Lemma 2 shows that the bagends of the children are disjoint except for the vertices in the MERGE bag. In particular, the only arcs that are considered in both bagends are the arcs in the current bag, which do not carry any flow. So the flow functions are essentially disjoint, and we can add them without violating capacity constraints. But we do have to ensure that no vertex in the current bag has outgoing arcs into both bagends.

Lemma 6. Let i be a MERGE node, and let j and k be the children of i , implying that $B_i = B_j = B_k$. Let $o, d^+ \in \{0, \dots, U\}^{B_i}$.

Then $(o, d^+) \in \text{TABLE}(i)$ if and only if there exist $(\bar{o}, \bar{d}^+) \in \text{TABLE}(j)$ and $(\hat{o}, \hat{d}^+) \in \text{TABLE}(k)$ such that $o = \bar{o} + \hat{o}$ and $d^+ = \bar{d}^+ + \hat{d}^+$, and for all $v \in B_i$ at least one of $\bar{o}(v)$ and $\hat{o}(v)$ is zero.

The remaining and most intricate case is that of a FORGET node. What makes this case so intricate is that essentially two steps happen at once. First, a set of arcs between the leaving vertex and the remaining vertices of the bag is chosen, and flow is pushed along these arcs, for which target outflow values have to be chosen. The additional supplies are adjusted accordingly as well. But then, the vertex is removed and none of these arcs are visible anymore. Below, we capture the amount of flow towards the forgotten vertex as the sum over the remaining vertices in y_{d^+} . The flow value out of the forgotten vertex to the remaining bag is the similar sum in y_o , which can have at most one non-zero term though.

Lemma 7. Let i be a FORGET node, and let j be the unique child of i . Let v be the leaving vertex, such that $B_i = B_j \setminus \{v\}$. Let $o, d^+ \in \{0, \dots, U\}^{B_i}$.

Then $(o, d^+) \in \text{TABLE}(i)$ if and only if there exists $(\bar{o}, \bar{d}^+) \in \text{TABLE}(j)$ such that the following hold.

1. For all $w \in B_i$ with $o(w) \neq \bar{o}(w)$, it holds that $\bar{o}(w) = 0$, and there is an arc $a = (w, v) \in A$ with $o(w) \leq u(a)$.
2. There is at most one vertex $w \in B_i$ with $d^+(w) \neq \bar{d}^+(w)$. If there is such a vertex w , then $\bar{o}(v) = 0$ and $d^+(w) > \bar{d}^+(w)$ must hold. Then there also is an arc $a = (v, w) \in A$ with $d^+(w) - \bar{d}^+(w) \leq u(a)$.
3. Let $y_o := \bar{o}(v) + \sum_{w \in B_i} (d^+(w) - \bar{d}^+(w))$ and $y_{d^+} := \bar{d}^+(v) + \sum_{w \in B_i} (o(w) - \bar{o}(w))$. Then $0 \leq y_o - y_{d^+} \leq d(v)$ must hold.

This completes the construction of the tables from a mathematical point of view. We now briefly discuss the running times of these constructions. Let p be a bound on the size of $\text{TABLE}(i)$ as an array, which is polynomial in U if the treewidth is bounded by a constant. For a LEAF node, we just need to set a single entry. This takes $\mathcal{O}(1)$ operations. For an INTRODUCE node i , we can loop over all entries in the child node j , extend them with $(0, 0)$ for the new vertex, and mark this new entry in $\text{TABLE}(i)$. This takes $\mathcal{O}(p)$ operations. For a MERGE node i , we can loop over all combinations of entries in the children j_1 and j_2 , check each combination with $\mathcal{O}(k) = \mathcal{O}(1)$ arithmetic operations, and, if applicable, mark an entry in $\text{TABLE}(i)$. This requires $\mathcal{O}(p^2)$ operations. For a FORGET node i with child j , we can consider each pair (o, d^+) , and check $\text{TABLE}(j)$ for a corresponding entry. This takes $\mathcal{O}(p^2)$ time. All together, this yields the following lemma.

Lemma 8. There is an algorithm that, given a nice tree decomposition $(I, \mathcal{T})$, computes all tables correctly and runs in time polynomial in $|I|$ and U , assuming that the width k of the tree decomposition is a constant.

3.4. Deriving the approximation scheme

We can now apply the recent Lemma 8 to solve the MAXCONF problem as follows. This also includes the construction to deal with multiple sinks that have limited demands.

Theorem 3. MAXCONF can be solved in pseudo-polynomial time if the treewidth of the graph is bounded by some constant.

Proof. Consider an instance with a digraph $G = (V, A)$, capacities $u : A \rightarrow \mathbb{Z}_{\geq 0}$, and supply/demand function $d : V \rightarrow \mathbb{Z}$, with sources S^+ and sinks S^- . Assume that the treewidth of G is at most some constant k .

We introduce a supersink t to obtain a graph G' . Notice that this preprocessing does not change the value of MAXCONF. Also, the treewidth of this new instance is at most $k + 1$, because a tree decomposition of G can be turned into a tree decomposition of G' by adding t to every bag.

Let $U := \sum_{v \in S^+} d(v)$. Since U is a bound on the MAXCONF value, we can reduce all capacities to U without changing the solution value. If an arc carried higher flow values, this would imply that flow units cycle in the graph, because they cannot reach the sink. Now the instance is suitable for Definition 7.

Next, we compute a nice tree decomposition of G' with t as the sink. This decomposition can be chosen to have width at most $k + 1$ and polynomially many nodes, as described in Lemma 1. Finally, we run the dynamic program from Lemma 8 on the modified instance to compute $\text{TABLE}(r)$ for $B_r = \{t\}$.

As we have already shown in Lemma 3, we can output the maximum value of $d^+(t)$ in $\text{TABLE}(r)$ as the solution value.

Note that the overall running time is polynomial in the size of G and U . $\square$

A theoretical advantage of this result is that we can now be certain that MAXCONF with multiple sinks on trees is *weakly* $\mathcal{NP}$ -hard, but no harder. More importantly, the result above yields a fully polynomial-time approximation scheme for MAXCONF, still under the assumption of bounded treewidth.

Theorem 4. MAXCONF on digraphs with $|V|$ vertices and treewidth bounded by some constant can be approximated to a factor of $(1 - \varepsilon)$ for any $\varepsilon > 0$ in time polynomial in $|V|$ and $1/\varepsilon$.

The approximation can be obtained by a simple rounding scheme. For the proof, we refer to [7]. As usual, backtracking in the dynamic program provides the actual flow as well for both algorithms. Note that it is also possible to modify the exact dynamic program in order to handle, for example, lower capacities on arcs or all-or-nothing flows. However, since the accuracy of the rounded instance is limited, these conditions only carry over in an $\mathcal{O}(\varepsilon)$ -precise way to the approximation algorithm.

An additional point to consider is the practicality of these algorithms. Even the running time of the FPTAS suffers from a large exponent depending on the treewidth. It might just be that the dynamic program as written is only suitable for graphs with treewidth 2. At least this covers approximating MAXCONF on trees with multiple sinks.

4. Outerplanar graphs

4.1. Zigzag paths on outerplanar graphs instances

One observation encourages us to reconsider the gap between easy and hard confluent flow problems despite the hardness results on trees. On cycles with a single sink, MAXCONF can be solved efficiently. There are only $|V|$ possibilities to split the flow: the flow on one side of this split point reaches the sink clockwise, the flow on the other side moves counterclockwise.

We will now extend this approach to outerplanar graphs, which look “somewhat like cycles” but are complex enough to foil a brute-force approach. The ideas presented in this section fall into two groups. First, we explore the hierarchy of a certain class of self-contained induced subgraphs to enable dynamic programming, which is a common approach for outerplanar graphs. Second, we analyze and exploit the properties of the dynamic programming solutions to achieve polynomial running times by expressing the data in a compressed way.

Definition 8 (*Outerplanar Graph*). An undirected graph $G = (V, E)$ is called *outerplanar* if it has an embedding in the plane such that all vertices lie on the outer face. We will call a digraph G outerplanar if the underlying undirected graph $\bar{G}$ is outerplanar.

Outerplanar graphs are exactly those graphs that do not have K_4 or $K_{2,3}$ as minor [4], and they can be recognized in linear time [12]. The usual drawing of an outerplanar graph places all vertices on a circle, and all edges are embedded within that circle.

We start with a high-level description of the algorithm. As mentioned above, we can assume that we are given a straight-line embedding of a bidirected outerplanar graph $G = (V, A)$ on the regular n -gon, and that the vertices are labeled clockwise v_0 to v_{n-1} . We may also assume that the entire outer cycle is part of the graph in both directions. Adding these arcs with capacity 0 cannot lead to any crossings, by simple geometric arguments. The (unlimited) sink is v_0 , located at the bottom of the cycle, and we think of vertices with low indices as being left of vertices with higher indices (see Fig. 4).

Our dynamic program works on “slices” cut out of the cycle. More precisely, for $i < j$, we consider the graph induced by $\{v_i, v_{i+1}, \dots, v_j\}$, which we denote by $G_{i,j}$. This is again an outerplanar graph (by the same embedding as G). We can assume that $G_{i,j}$ is self-contained, that is, no vertex v_k with $i < k < j$ is incident to a vertex outside of $G_{i,j}$. We want to capture all the relevant information about the flow inside $G_{i,j}$ in the dynamic program. Since $G_{i,j}$ is self-contained, we only need to know the balance at v_i and v_j . Once this information is available, we will join consecutive $G_{i,j}$ and $G_{j,k}$ to obtain the values for $G_{i,k}$.

To determine the balance of v_i and v_j for the dynamic program, consider an optimum confluent flow on G . The flow-carrying arcs induce an in-tree pointing towards the sink which we can greedily extend to a spanning in-tree. The path from any vertex in $G_{i,j}$ to v_0 must travel through either v_i or v_j . This gives rise to three cases.

The subgraph $G_{i,j}$ could be split by the tree into two components which must then be trees rooted at v_i and v_j . We denote this case by $\text{SPLIT}_{i,j}$. To compute the corresponding entries in the dynamic program, we can treat v_i and v_j as sinks in a smaller confluent flow instance consisting only of $G_{i,j}$. Otherwise, the tree induces only one component in $G_{i,j}$ which is either rooted

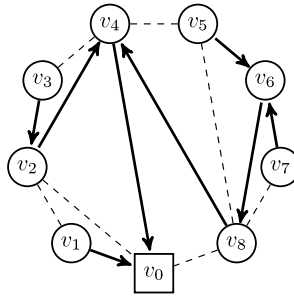


Fig. 4. A maximal outerplanar graph and a possible spanning in-tree. An outerplanar graph consists of up to $2|V| - 3$ edges (or $4|V| - 6$ arcs, respectively), and therefore contains an exponential number of spanning in-trees.

at v_i or v_j . If it is rooted at v_i , then we say the flow is routed left, and treat v_i as a sink. This is the case $\text{LEFT}_{i,j}$. But it may be that additional flow enters $G_{i,j}$ at v_j in an optimum solution. Therefore we have to treat v_j as a source with *undetermined supply*. For a dynamic program, we therefore have to know the flow value at v_i for all possible supplies of v_j . In the other case, called $\text{RIGHT}_{i,j}$, the flow leaves $G_{i,j}$ through v_j , which acts as a sink, while v_i is a variable source.

The dynamic program needs a separate table for each of the three cases. For $\text{SPLIT}_{i,j}$, all dominating pairs of flow values arriving at v_i and v_j are computed. There can be only $(j - i)$ such values, as these situations can be characterized by a split point between v_i and v_j .

For $\text{LEFT}_{i,j}$ (and analogously for $\text{RIGHT}_{i,j}$), the desired table is a function describing the maximum flow that can reach the temporary sink v_i for each possible amount of supply at v_j , and not just for $d(v_j)$. Without further effort, these tables would have a pseudo-polynomial size. However, this function is what we want to call *simple*. For every unit of additional supply at v_j , the output at v_i can increase by 1 or remain unchanged. We will bound the number of times this behavior changes by $(j - i)^2$. If we additionally perform the entire computations with such a piecewise description of the tables in mind, we obtain a polynomial running time.

To have a more symmetric notation, we actually want to split up the single uncapacitated sink v_0 into two sinks. We introduce a new node v_n between v_0 and v_{n-1} , and make it a second sink. The capacities between v_{n-1} and v_n are inherited from the former arcs (v_{n-1}, v_0) and (v_0, v_{n-1}) , but there is no arc between v_0 and v_n . All the chords to and from v_0 remain at v_0 . Both sinks again have “infinite” demand. One can easily check that the original instance has the same MAXCONF value as the new instance. We will further assume that these sinks use no outgoing arc, because they have infinite demands. These preparations allow us to focus on the following kind of instance.

Definition 9 (*Outerplanar Instance*). An outerplanar instance consists of an outerplanar graph $G = (V, A)$ with the vertices $V = \{v_0, \dots, v_n\}$ embedded on the regular $(n + 1)$ -gon and labeled clockwise, arc capacities u , supplies d , and sinks v_0 and v_n with infinite demand. Furthermore, all arcs along the outer cycle except (v_0, v_n) and (v_n, v_0) must be in A .

The following definitions formalize the dynamic programming tables.

Definition 10. Consider an outerplanar instance and $i, j \in \mathbb{N}_0$ with $0 \leq i < j \leq n$.

- Let $G_{i,j} := G[\{v_i, v_{i+1}, \dots, v_j\}]$.
- Let $\text{LEFT}_{i,j}(z) : \mathbb{N}_0 \rightarrow \mathbb{N}_0$ be the maximum value of a confluent flow in $G_{i,j}$ with capacities u and supplies d , except $d(v_i) := -\infty$ and $d(v_j) := z$.
- Let $\text{LEFT}_{i,i}(z) := 0$ for all $z \in \mathbb{N}_0$.
- Let $\text{RIGHT}_{i,j}(z) : \mathbb{N}_0 \rightarrow \mathbb{N}_0$ be the maximum value of a confluent flow in $G_{i,j}$ with capacities u and supplies d , except $d(v_i) := z$ and $d(v_j) := -\infty$.
- Let $\text{RIGHT}_{i,i}(z) := 0$ for all $z \in \mathbb{N}_0$.
- Let $\text{SPLIT}_{i,j} \subseteq \mathbb{N}_0^2$ contain exactly those pairs (l_k, r_k) with $l_k = \text{LEFT}_{i,k}(d_k)$ and $r_k = \text{RIGHT}_{k+1,j}(d_{k+1})$ for $k \in \{i, \dots, j-1\}$.

Fig. 5 shows an example why the RIGHT functions for larger subgraphs are not trivial. The crucial observation is that more flow entering on the left does not increase the output on the right immediately, but at some point the optimal solution might switch to an in-tree that allows for more capacity on the chord, but routes less of the flow originating inside $G_{i,j}$ to the right, still resulting in a net gain. On the other hand, if the chord is not needed to route anything past $G_{i,j}$, it is usually more effective to route the flow from vertices on the left through the chord instead of along the outer cycle. Finding the right balance between capacity on the chord and sending flow from inside to the outside is the trick. Note that this leads to optimal solutions that send flow in a zigzag path to the sink, which is quite unlike the flow in a cycle.

We now show the reasoning behind the definition of $\text{SPLIT}_{i,j}$ and how we can compute the optimum MAXCONF value from $\text{SPLIT}_{0,n}$. The key property of the outerplanar embedding is that, if we consider two undirected chords $\{v_i, v_j\}$ and $\{v_k, v_l\}$, with $i < j$ and $i < k < l$, these chords cross if and only if $i < k < j < l$. This can be extended to paths. If there is a path from v_j to v_i (going left) and from v_l to v_k , then these paths must have a vertex in common if $i < k < j < l$. Thus, there always

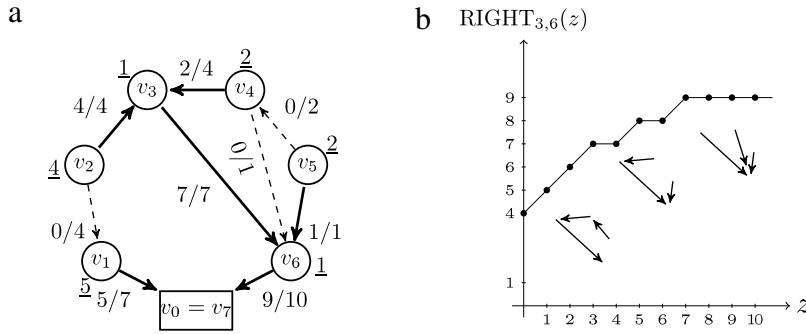


Fig. 5. (a) The supplies d are underlined, and the optimal solution is written as x/u on the arcs. (b) The plot of $\text{RIGHT}_{3,6}(z)$ and the defining in-trees below.

is a vertex v_m with the highest index that sends flow to v_0 in an optimum solution, and no flow is sent between $G_{0,m}$ and $G_{m+1,n}$. This gives us our next lemma.

Lemma 9. Consider an outerplanar instance. Let x be a confluent flow for this instance. W.l.o.g., assume that v_0 and v_n use no outgoing arc. Then there exists some $m \in \{0, \dots, n-1\}$ such that all flow-carrying arcs F_x lie in $A(G_{0,m}) \cup A(G_{m+1,n})$.

Proof. For $i \in \{0, 1, \dots, n\}$, let P_i be the path from v_i to either v_0 or v_n in F_x , or $P_i = \emptyset$ if there is no path from v_i to the sinks in F_x . Let $m := \max\{i : 0 \leq i \leq n-1 \wedge \emptyset \neq V(P_i) \subseteq \{v_0, v_1, \dots, v_i\}\}$. Since $i = 0$ is always admissible, m is properly defined. We claim that this is the desired index.

Suppose that there exists an arc in $F_x \setminus A(G_{0,m}) \cup A(G_{m+1,n})$. We need to treat the two directions of this arc slightly differently.

If there is an arc (v_i, v_j) from $G_{0,m}$ to $G_{m+1,n}$, then P_m cannot contain v_i , which would imply containing v_j as well, because P_m does not contain any vertex with a larger index than m . Then there must be a chord (v_k, v_l) in P_m with $l < i < k \leq m < j$ that enables P_m to bypass v_i . However, (v_k, v_l) would cross the chord (v_i, v_j) .

Now that we have established that no chord carries flow from $G_{0,m}$ to $G_{m+1,n}$, we consider the case that there is a chord (v_j, v_i) carrying flow in the other direction. Since v_n has no outgoing flow, $j < n$ must hold. This implies that $V(P_j) = V(P_i) \cup \{v_j\}$, and P_i cannot use any chord going back to $G_{m+1,n}$, as just shown. This makes $j > m$ eligible for the choice of m , a contradiction. $\square$

Given $\text{SPLIT}_{0,n}$, it is now trivial to find the optimum MAXCONF value.

Lemma 10. The optimum MAXCONF value of an outerplanar instance is

$$\max_{(l,r) \in \text{SPLIT}_{0,n}} (l + r).$$

Proof. Choose (l, r) that attain the maximum. By the definition of $\text{SPLIT}_{0,n}$, this implies the existence of two confluent flows on the disjoint subgraphs $G_{0,k}$ and $G_{k+1,n}$, with sinks v_0 and v_n , respectively, and with the original demands. Thus, we can easily combine them to obtain a feasible solution. (Note that, for $k = 0$ or $k = n-1$, one of the subgraphs consists of a single vertex and no arc. Thus, the flow is actually an empty function. This is not a problem, though, as $\text{LEFT}_{i,i}$ and $\text{RIGHT}_{i,i}$ are always 0.)

On the other hand, Lemma 9 shows that for any confluent flow x there always is an index m where the outerplanar graph is split by F_x . Plugging m into the definition of $\text{SPLIT}_{0,n}$, we see that the optimum solution gives an entry $(l^*, r^*) \in \text{SPLIT}_{0,n}$ with l^* and r^* being the amount of flow sent to the two sinks. Then $l + r \geq l^* + r^*$ must be true, but the latter is the optimum value. $\square$

4.2. Computing the throughput functions

We will now show how one can compute the functions LEFT and RIGHT efficiently in a dynamic program, which immediately implies the computation for SPLIT as well. As mentioned above, we compute the values for $G_{i,j}$ from self-contained subgraphs $G_{i,m}$ and $G_{m,j}$. We need to ensure that this is always possible.

Lemma 11. Consider an outerplanar instance. For $j - i \geq 2$, $0 \leq i < j \leq n$, there always exists $m \in \{i+1, \dots, j-1\}$ such that there is no arc in G between $\{v_i, \dots, v_{m-1}\}$ and $\{v_{m+1}, \dots, v_j\}$, besides maybe (v_i, v_j) and (v_j, v_i) .

Proof. Let $j - i \geq 2$. If v_i has no incoming arc from or outgoing arc to $\{v_{i+1}, \dots, v_{j-1}\}$, then simply choose $m := i+1$. This already fulfills the statement of the lemma.

Otherwise, let $m := \max\{m' : i < m' < j \wedge ((v_i, v_{m'}) \in A \vee (v_{m'}, v_i) \in A)\}$. This is now well defined. Any arc between a vertex in $\{v_{i+1}, \dots, v_{m-1}\}$ and a vertex in $\{v_{m+1}, \dots, v_j\}$ would be a chord, because it bypasses v_m . Then such a chord would also cross the chord (i, m) or (m, i) , which is a contradiction to the assumed embedding of G . Any other arc which the lemma forbids would have to be incident to v_i . Then it either contradicts the choice of m or is actually a chord between v_i and v_j , which is allowed. $\square$

Lemma 12. Consider an outerplanar instance. Let $0 \leq i < m < j \leq n$ be as in Lemma 11, that is, $G_{i,m}$ and $G_{m,j}$ are self-contained. Assume that the chords (v_i, v_j) and (v_j, v_i) exist but maybe with capacity 0.

1. Then $\text{LEFT}_{i,i+1}(z) = \min\{u(v_{i+1}, v_i), z\}$ holds.
2. Then $\text{RIGHT}_{i,i+1}(z) = \min\{u(v_i, v_{i+1}), z\}$ holds.
3. Define

$$\text{CHORDLEFT}_{i,j}(z) := \max_{(l,r) \in \text{SPLIT}_{i,j}} l + \min\{u(v_j, v_i), z + r\}.$$

Then

$$\text{LEFT}_{i,j}(z) = \max\{\text{LEFT}_{i,m}(\text{LEFT}_{m,j}(z) + d(m)), \text{CHORDLEFT}_{i,j}(z)\}$$

holds.

4. Define

$$\text{CHORDRIGHT}_{i,j}(z) := \max_{(l,r) \in \text{SPLIT}_{i,j}} r + \min\{u(v_i, v_j), z + l\}.$$

Then

$$\text{RIGHT}_{i,j}(z) = \max\{\text{RIGHT}_{m,j}(\text{RIGHT}_{i,m}(z) + d(m)), \text{CHORDRIGHT}_{i,j}(z)\}$$

holds.

Statements 1 and 2 are obvious. For 3 and 4, we do a case analysis, whether $(v_j, v_i) \in F_x$, but we omit the details here. They can be found in [13]. However, using these formulas to evaluate the functions at certain values of z is not advisable in a polynomial algorithm. Computing $\text{LEFT}_{i,j}$ or $\text{RIGHT}_{i,j}$ in the interesting case where there is a chord (i, j) or (j, i) with positive capacity relies on $\text{SPLIT}_{i,j}$. This again relies on LEFT and RIGHT evaluations for $j - i$ subgraphs, and so on. In particular, the same pair of indices might occur in different subtrees of the recursion. So we want to use a dynamic programming approach instead, which computes the required values in a bottom-up way. There is one big problem, though, compared to the recursion: we do not know for which values of z we need to evaluate the LEFT and RIGHT functions, so we will compute the *entire* functions! Done naïvely, this only yields a pseudo-polynomial algorithm, because z may take pseudo-polynomially many values. However, we can describe these functions efficiently, making use of their simple “staircase” form as seen in Fig. 5(b). Even better, we can carry out the computations in Lemma 12 on these efficient descriptions much more quickly than for all values of z by themselves.

Definition 11 (Simple Function). Let $f : \mathbb{N}_0 \rightarrow \mathbb{N}_0$ be a function such that $f(z + 1) \in \{f(z), f(z) + 1\}$ for all $z \in \mathbb{N}_0$. Then we call f a simple function. Such a function can be described by $f(0)$ and the intervals on which f increases. We denote the size of this set of intervals by $\langle f \rangle$.

Note that it is easy to store the $\langle f \rangle$ intervals describing f in a linked list. This trivially allows the evaluation of $f(z)$ in $\mathcal{O}(\langle f \rangle)$.

Lemma 13. $\text{LEFT}_{i,j}$ and $\text{RIGHT}_{i,j}$ are simple.

Proof. As $\text{LEFT}_{i,j}$ describes a maximum flow value, $\text{LEFT}_{i,j}(z + 1) \geq \text{LEFT}_{i,j}(z)$, because the flow cannot become worse with higher supply by using the same in-forest F_x of flow-carrying arcs. Also $\text{LEFT}_{i,j}(z + 1) \leq \text{LEFT}_{i,j}(z) + 1$ holds, because any in-forest used for $\text{LEFT}_{i,j}(z + 1)$ could have been used for $\text{LEFT}_{i,j}(z)$, losing at most one unit of flow. $\text{RIGHT}_{i,j}$ is simple by the same arguments. $\square$

We can operate on simple functions as follows.

Lemma 14. Let f_1 and f_2 be two simple functions.

1. $g(z) := f_1(f_2(z))$ is simple and $\langle g \rangle \leq \langle f_1 \rangle + \langle f_2 \rangle$.
2. $g(z) := \max\{f_1(z), f_2(z)\}$ is simple and $\langle g \rangle \leq \langle f_1 \rangle + \langle f_2 \rangle$.
3. For $c \in \mathbb{N}_0$, $g(z) := f_1(z) + c$ is simple and $\langle g \rangle = \langle f_1 \rangle$.
4. For $c \in \mathbb{N}_0$, $g(z) := f_1(z + c)$ is simple and $\langle g \rangle \leq \langle f_1 \rangle$.

In all cases, g can be computed in time linear in $\langle f_1 \rangle$ and $\langle f_2 \rangle$, where applicable.

The claims of Lemma 14 follow by some simple calculus. We can now apply these considerations to the recursive definitions in Lemma 12.

Lemma 15. For all $0 \leq i \leq j \leq n$, the following hold.

1. $|\text{SPLIT}_{i,j}| \leq j - i$ (assuming $i < j$).
2. $\langle \text{LEFT}_{i,j} \rangle \leq (j - i)^2$.
3. $\langle \text{RIGHT}_{i,j} \rangle \leq (j - i)^2$.

Proof. 1. All the entries in $\text{SPLIT}_{i,j}$ are determined by one of the values of $k \in \{i, \dots, j - 1\}$.

2. We use induction on $j - i$. For $j - i = 0$, the claim holds because $\text{LEFT}_{i,i} \equiv 0$ and $\langle \text{LEFT}_{i,i} \rangle = 0$. Similarly elemental, for $j - i = 1$, we note that $\langle \min\{u_{v_j, v_i}, z\} \rangle \leq 1$.

Induction and Lemma 14 yield $\langle \text{LEFT}_{i,m}(\text{LEFT}_{m,j}(z) + d(m)) \rangle \leq (j - m)^2 + (m - i)^2$.

Now consider $\text{CHORDLEFT}_{i,j}$. It is the maximum over $|\text{SPLIT}_{i,j}| \leq j - i$ simple functions of size at most 1. So $\langle \text{CHORDLEFT}_{i,j} \rangle \leq j - i$. This plus the size of $\text{LEFT}_{i,m}(\text{LEFT}_{m,j}(z) + d(m))$ bounds the size of $\text{LEFT}_{i,j}$ due to the computation by a maximum over the two. But $(j - m)^2 + (m - i)^2 + j - i \leq (j - m)^2 + (m - i)^2 + 2(j - m)(m - i)$, because $i < m < j$, and the latter is simply $(j - i)^2$.

3. As above. $\square$

Now all the pieces for a polynomial algorithm for MAXCONF on outerplanar graphs with one sink are set up.

Theorem 5. MAXCONF on an outerplanar graph G with a single sink can be solved in polynomial time.

Proof. First, find an embedding of G , and transform the input into an outerplanar instance. Then, starting with $j - i = 0$ and proceeding towards $j - i = n$, compute all $\text{SPLIT}_{i,j}$, then $\text{LEFT}_{i,j}$ and $\text{RIGHT}_{i,j}$ inductively in a dynamic program. For each of these pairs (i, j) , $\text{SPLIT}_{i,j}$ can be computed from various $\text{LEFT}_{i,k}$ and $\text{RIGHT}_{k+1,j}$, with $k - i < j - i$ and $j - (k + 1) < j - i$. Having done so, compute $\text{LEFT}_{i,j}$ from $\text{SPLIT}_{i,j}$, $\text{LEFT}_{i,m}$ and $\text{LEFT}_{m,j}$ with $i < m < j$, and similarly for $\text{RIGHT}_{i,j}$. Importantly, all of these calculations are performed on the interval representation of the simple functions.

Lemma 15 shows that at no time can the size of SPLIT be larger than $j - i$, while LEFT and RIGHT need at most $(j - i)^2$ entries to describe them. Computing each $\text{SPLIT}_{i,j}$ requires $\mathcal{O}(n)$ evaluations of LEFT and RIGHT functions, each of which can be done in $\mathcal{O}(\langle \text{LEFT} \rangle) = \mathcal{O}(n^2)$ time. So, for each $\text{SPLIT}_{i,j}$, we need at most $\mathcal{O}(n^3)$ time. Computing LEFT requires 2 evaluations of LEFT and the maximum over $\mathcal{O}(n)$ entries in SPLIT. So this takes $\mathcal{O}(n^2)$ time. Note that we can find the splitting point m with a simple greedy strategy by checking the existence of $\mathcal{O}(n)$ arcs, as shown in the proof of Lemma 12. Clearly, this is dominated by $\mathcal{O}(n^3)$. So for each pair (i, j) we can find all necessary functions in $\mathcal{O}(n^3)$ time, and the entire table is computed in $\mathcal{O}(n^5)$ time. Finally, the maximum value can be found in $\mathcal{O}(n)$ time from $\text{SPLIT}_{0,n}$, as noted in Lemma 10. $\square$

While the running time might not seem too good, here are a few things to consider. So far, we do not know of any instance where LEFT and RIGHT actually have more than linearly many intervals. Maybe one can tighten the analysis of Lemma 15, which would immediately imply a faster running time. Furthermore, a better algorithm is already possible, because not all pairs (i, j) are actually required to compute $\text{SPLIT}_{0,n}$. Indeed, the costly computation of $\text{SPLIT}_{i,j}$ is only needed if there is a chord (v_i, v_j) or (v_j, v_i) , because otherwise CHORDLEFT is not a good option. But there are only $\mathcal{O}(n)$ chords in an outerplanar graph (actually, any planar graph has only $\mathcal{O}(n)$ edges), so an improved algorithm would need to construct $\text{SPLIT}_{i,j}$ only for $\mathcal{O}(n)$ pairs of indices. This implies a running time of $\mathcal{O}(n^4)$ that could already be realized.

4.3. Efficient algorithms for confluent flows and forbidden minors

On the one hand, outerplanar graphs are exactly those graphs that do not contain a K_4 or a $K_{2,3}$ as a minor [4]. On the other hand, the instance in Fig. 2 uses a $K_{2,k}$ to model PARTITION, but is K_4 -minor free. This encourages us to study minors and their influence on the computation of maximum confluent flows. In this subsection, we will extend the results from outerplanar graphs to $K_{2,3}$ -minor-free graphs, and we will formulate a conjecture for $K_{2,k}$ -minor-free graphs for arbitrary k . We start with two observations. Again, we omit some technical details. The missing proofs in this subsection can be found in [13].

Lemma 16. Let G be a (undirected) graph which does not contain the complete bipartite graph $K_{2,3}$ as a minor. If G contains the complete graph K_4 as a minor, then this K_4 is an induced subgraph of G .

Lemma 17. Let G be a (undirected) graph which does not contain the complete bipartite graph $K_{2,3}$ as a minor. Furthermore, let G contain K_4 as an induced subgraph. Let v and w be two arbitrary vertices of this induced K_4 . Then there is no path between v and w which contains any other vertices than the nodes of this K_4 .

With these two lemmata, it is easy to understand the structure of a graph that contains no $K_{2,3}$ as a minor. The graph may contain some induced K_4 subgraphs. Deleting all edges of these K_4 subgraphs, the graph decomposes into outerplanar components. Lemma 17 ensures that two induced K_4 or an induced K_4 and an induced outerplanar subgraph have at most one common vertex. Furthermore, it guarantees a tree structure of these components, since there are no paths connecting two nodes of a K_4 . The underlying tree H can be constructed easily. For each induced K_4 and each induced maximal outerplanar component, add one vertex. Connect two nodes if and only if the corresponding components have a common vertex.

H is similar to the block graph of G . Note that two nodes corresponding to outerplanar components are never connected directly, since we consider only maximal outerplanar components. Thus, each cycle in H would contain at least one node corresponding to a K_4 . Therefore, a cycle would induce a forbidden path between two nodes of the K_4 . Hence, H contains no cycles. We can use this underlying tree structure to solve MAXCONF on graphs without $K_{2,3}$ minor.

Corollary 1. *If G contains no $K_{2,3}$ as a minor, then MAXCONF with a single sink can be solved in polynomial time.*

Proof. We exploit the underlying tree structure. First, construct H by decomposing G into maximal outerplanar components and K_4 components. H is a tree, thus the direction of flow is well defined on H and therefore on each component. W.l.o.g., the sink is contained in only one component, which corresponds to the root of H (add a new sink and connect it to the old one via one arc). The components have at most one common vertex. These vertices act as temporary sources or sinks, and we compute a maximum confluent flow for each component starting at the leaves of H . The incoming flow of a temporary sink defines the maximum supply of the corresponding source in the parent component. This implies that the maximum value of a confluent flow in G is equal to the maximum value of a confluent flow in the root component. Maximum values of confluent flows on outerplanar components are processed as in Theorem 5. K_4 components have fixed size; therefore a maximum confluent flow can be found in $\mathcal{O}(1)$ time. Obviously, the number of components is bounded by the size of G . Thus, the claim follows. $\square$

Note that there is no polynomial-time algorithm for the class of $K_{3,3}$ -minor-free graphs (unless $\mathcal{P} = \mathcal{NP}$), as this allows a $K_{2,k}$, which can again model a PARTITION instance (see Fig. 2). However, there is room for one stronger conjecture.

Conjecture 1. *For any constant $k \in \mathbb{N}$, MAXCONF on $K_{2,k}$ -minor-free graphs with a single sink can be solved in polynomial time.*

A graph that is $K_{2,k}$ -minor free is probably too restrictive to contain arbitrarily large $\mathcal{NP}$ -hard subproblems like PARTITION. In particular, this weaker requirement could also allow outerplanar graphs with up to $k - 1$ sinks modeled by a supersink. However, an algorithm for this problem might be quite involved.

5. Conclusion

In this paper, we have studied special cases of the confluent flow problem. We have demonstrated an exact pseudo-polynomial-time algorithm and an FPTAS for MAXCONF on graphs with bounded treewidth which matched with our hardness results derived earlier. Furthermore, we have presented a polynomial-time algorithm for outerplanar graphs.

Future research should look into approximation algorithms (not schemes, as those cannot exist) for MAXCONF on general graphs or improve the inapproximation bound from [8]. Maybe one can even show that MAXCONF cannot be approximated within a constant factor. Restricted to uniform capacities, this is unlikely because of the 13.29-approximation by Chen et al. [5] for the all-or-nothing variant of MAXCONF. Finally, one may shorten or even close the gap between easy and hard confluent flow problems. One could start with a proof of Conjecture 1.

References

- [1] Stefan Arnborg, Jens Lagergren, Detlef Seese, Easy problems for tree-decomposable graphs, *J. Algorithms* 12 (1991) 308–340.
- [2] Andreas Bley, Routing and Capacity Optimization for IP Networks, Ph.D. Thesis, TU Berlin, 2007.
- [3] Hans Bodlaender, Treewidth: algorithmic techniques and results, in: 22nd International Symposium on Mathematical Foundations of Computer Science, Springer, 1997, pp. 29–36.
- [4] Gary Chartrand, Frank Harary, Planar permutation graphs, *Ann. Inst. H. Poincaré (Section B)* 3 (4) (1967) 433–438.
- [5] Jianghuo Chen, Robert D. Kleinberg, László Lovász, Rajmohan Rajaraman, Ravi Sundaram, Adrian Vetta, (Almost) tight bounds and existence theorems for single-commodity confluent flows, *J. ACM* 54 (4) (2007).
- [6] Jianghuo Chen, Rajmohan Rajaraman, Ravi Sundaram, Meet and merge: approximation algorithms for confluent flows, *J. Comput. System Sci.* 72 (3) (2006) 468–489.
- [7] Daniel Dressler, Network Flow Problems Arising From Evacuation Planning, Ph.D. Thesis, TU Berlin, Berlin, Germany, 2012. <http://opus.kobv.de/tuberlin/volltexte/2012/3683/>.
- [8] D. Dressler, M. Strehler, Capacitated confluent flows: complexity and algorithms, in: Proceedings of the 7th International Conference on Algorithms and Complexity, CIAC 2010, in: Lecture Notes in Computer Science, vol. 6078, Springer, Heidelberg, 2010, pp. 347–358.
- [9] Lester R. Ford, Delbert R. Fulkerson, Maximal flow through a network, *Canad. J. Math.* 8 (1956) 399–404.
- [10] Armin Fügenschuh, Henning Homfeld, Hanno Schueldorf, Routing cars in rail freight service, in: Dagstuhl Seminar Proceedings 09261, Dagstuhl, Germany, 2009.
- [11] Satoru Fujishige, Kazuhisa Makino, Satoko Mamada, Takeaki Uno, A tree partitioning problem arising from an evacuation problem in tree dynamic networks, *J. Oper. Res. Soc. Japan* 48 (3) (2005) 196–206.
- [12] S.L. Mitchell, Linear algorithms to recognize outerplanar and maximal outerplanar graphs, *Inform. Process. Lett.* 9 (1979) 229–232.
- [13] Martin Strehler, Signalized Flows—Optimizing Traffic Signals and Guideposts and Related Network Flow Problems, Ph.D. Thesis, Brandenburgische Technische Universität, Cottbus, Germany, 2012. <http://opus.kobv.de/btu/volltexte/2012/2363/>.